

tf.switch_case Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/switch_case

Create a switch/case operation, i.e.

Function Signatures

```
tf.switch_case( branch_index, branch_fns, default=None,
name='switch_case' )
```

Code Examples

```
tf.switch_case(
branch_index, branch_fns, default=None, name='switch_case'
)
```

```
tf.switch_case(
branch_index, branch_fns, default=None, name='switch_case'
)
```

```
switch (branch_index) { // c-style switch
case 0: return 17;
case 1: return 31;
default: return -1;
}
```

```
switch (branch_index) { // c-style switch
case 0: return 17;
case 1: return 31;
default: return -1;
}
```

```
branches = {0: lambda: 17, 1: lambda: 31}
branches.get(branch_index, lambda: -1)()
```

```
branches = {0: lambda: 17, 1: lambda: 31}
branches.get(branch_index, lambda: -1)()
```

```
def f1(): return tf.constant(17)
def f2(): return tf.constant(31)
def f3(): return tf.constant(-1)
r = tf.switch_case(branch_index, branch_fns={0: f1, 1: f2},
default=f3)
# Equivalent: tf.switch_case(branch_index, branch_fns={0: f1, 1:
f2, 2: f3})
```

```
def f1(): return tf.constant(17)
def f2(): return tf.constant(31)
def f3(): return tf.constant(-1)
r = tf.switch_case(branch_index, branch_fns={0: f1, 1: f2},
default=f3)
# Equivalent: tf.switch_case(branch_index, branch_fns={0: f1, 1:
f2, 2: f3})
```

Documentation

Create a switch/case operation, i.e.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.switch_case`

an integer-indexed conditional.

See also `tf.case`.

This op can be substantially more efficient than `tf.case` when exactly one branch will be selected. `tf.switch_case` is more like a C++ switch/case statement than `tf.case`, which is more like an if/elif/elif/else chain.

The `branch_fns` parameter is either a dict from int to callables, or list of (int, callable) pairs, or simply a list of callables (in which case the index is implicitly the key). The `branch_index` Tensor is used to select an element in `branch_fns` with matching int key, falling back to default if none match, or `max(keys)` if no default is provided. The keys must form a contiguous set from 0 to `len(branch_fns) - 1`.

`tf.switch_case` supports nested structures as implemented in `tf.nest`. All callables must return the same (possibly nested) value structure of lists,

tuples, and/or named tuples.

Example:

Pseudocode:

Expressions:

Returns

Raises